SMARTHOLOL

AI Solutions | Automation | Custom Development

TEST TECHNIQUE — STAGE DÉVELOPPEMENT WEB

Application de Gestion de Pharmacie

Stack Backend	Stack Frontend
Python / Django REST Framework PostgreSQL Swagger / drf-spectacular	React.js (Vite ou CRA) Axios / React Query Clean architecture & composants

Version	Rédigé par	Statut
1.0	Équipe Technique SMARTHOLOL	Confidentiel

1. À Propos de SMARTHOLOL

SMARTHOLOL est une société technologique spécialisée dans le développement de solutions numériques avancées, d'agents d'intelligence artificielle et de systèmes d'automatisation intelligents. Nous concevons et déployons des solutions sur mesure pour des clients au Maroc, aux États-Unis, en Australie et au Canada.

Notre mission est simple : transformer les processus métier de nos clients grâce à la puissance de l'IA et du développement moderne. Chaque projet que nous livrons respecte des standards de qualité élevés, en intégrant les meilleures pratiques de l'industrie et les technologies les plus récentes.

1.1 Notre Stack Technique

Nous travaillons principalement avec les technologies suivantes :

- Backend : Python, Django, Django REST Framework (DRF)
- Frontend : React.js, modern JS/TS ecosystem
- Bases de données : PostgreSQL, optimisé pour la performance
- IA & Agents : LLMs, RAG, automatisation intelligente
- Déploiement : Docker, CI/CD, cloud-native architecture
- Data : Web scraping, pipelines de données, ETL

1.2 Nos Valeurs de Développement

> *Chez SMARTHOLOL, nous croyons que la qualité du code est aussi importante que la fonctionnalité. Un stagiaire qui rejoint notre équipe adopte dès le premier jour nos pratiques : clean code, documentation systématique, tests, et collaboration via Git.*

Ce que nous valorisons	Ce que nous évitons
Clean code lisible et maintenable	Code spaghetti sans structure
Documentation Swagger complète pour chaque API	APIs non documentées
Commits Git clairs et conventionnels	Commits vagues ou massifs
Structure de projet claire et modulaire	Tout dans un seul fichier
Revues de code et collaboration	Travail en silos sans partage

© 2026 SMARTHOLOL — Tous droits réservés

2. Objectifs du Test Technique

Ce test technique a été conçu pour évaluer vos compétences réelles de développement full-stack dans un contexte professionnel. Il simule une situation de projet réel tel que nous le vivons chez SMARTHOLOL. Chaque livrable correspond à une pratique que nous appliquons quotidiennement.

#	Compétence évaluée	Ce que nous vérifions
1	Clean Code	Lisibilité, nommage, séparation des responsabilités, DRY, KISS
2	Documentation	Docstrings, commentaires pertinents, README structuré
3	Base de données	Modélisation claire, relations, migrations, contraintes d'intégrité
4	API REST	Endpoints RESTful, serializers, gestion des erreurs, status codes
5	Documentation Swagger	Schémas complets, descriptions, exemples de requêtes/réponses
6	Frontend React	Composants réutilisables, séparation UI/logique, appels API propres
7	Git / GitHub	Structure du repo, commits conventionnels, .gitignore, README

© 2026 SMARTHOLOL — Tous droits réservés

3. Présentation du Projet

3.1 Contexte

Vous êtes chargé de développer le MVP (Minimum Viable Product) d'une application web de gestion de pharmacie. Cette application permettra au pharmacien de gérer ses médicaments, de traiter les ventes, et d'être alerté quand les stocks sont bas.

> *Ce type d'application correspond exactement aux projets que nous développons pour nos clients : métier précis, données sensibles, besoin d'une API robuste et d'une UI claire. Votre façon de traiter ce projet nous donnera une image fidèle de votre façon de travailler.*

3.2 Nom du Projet

Nom du projet	PharmaManager — Système de gestion de pharmacie
Dépôt Git	pharma-manager (ou nom similaire clair)
Backend	Django REST Framework + PostgreSQL
Frontend	React.js (Vite recommandé)
Documentation API	Swagger via drf-spectacular

© 2026 SMARTHOLOL — Tous droits réservés

4. Cahier des Charges Fonctionnel

4.1 Gestion des Médicaments

Le système doit permettre la gestion complète du catalogue de médicaments de la pharmacie.

Fonctionnalités requises :

- Ajouter un médicament avec toutes ses informations
- Modifier les informations d'un médicament existant
- Supprimer un médicament (soft delete recommandé)
- Consulter la liste de tous les médicaments avec filtres et recherche
- Consulter le détail d'un médicament spécifique
- Alertes automatiques pour les médicaments dont le stock est inférieur au seuil minimum

Données d'un médicament :

Champ	Type	Description
nom	CharField	Nom commercial du médicament
dci	CharField	Dénomination Commune Internationale
categorie	ForeignKey	Catégorie (ex: antibiotique, antalgique...)
forme	CharField	Forme galénique (comprimé, sirop, injection...)
dosage	CharField	Ex: 500mg, 250mg/5ml
prix_achat	DecimalField	Prix d'achat unitaire
prix_vente	DecimalField	Prix de vente public
stock_actuel	IntegerField	Quantité disponible en stock
stock_minimum	IntegerField	Seuil d'alerte de réapprovisionnement
date_expiration	DateField	Date de péremption
ordonnance_requise	BooleanField	Médicament sous ordonnance ou non
date_creation	DateTimeField	Horodatage automatique (auto_now_add)
est_actif	BooleanField	Soft delete — défaut: True

4.2 Gestion des Ventes

Le système doit permettre d'enregistrer et de suivre les transactions de vente. Chaque vente peut contenir plusieurs médicaments.

Fonctionnalités requises :

© 2026 SMARTHOLOL — Tous droits réservés

- Créer une nouvelle vente avec un ou plusieurs articles
- Déduire automatiquement les quantités vendues du stock
- Consulter l'historique des ventes avec filtrage par date
- Annuler une vente (réintégration du stock, soft delete)
- Consulter le détail d'une vente avec tous ses articles

Structure de données pour les ventes :

Modèle	Champ	Description
Vente	reference	Code unique auto-généré (ex: VNT-2024-0001)
Vente	date_vente	Date et heure de la transaction
Vente	total_ttc	Montant total calculé automatiquement
Vente	statut	En cours / Complétée / Annulée
Vente	notes	Remarques optionnelles
LigneVente	vente	ForeignKey vers Vente
LigneVente	medicament	ForeignKey vers Medicament
LigneVente	quantite	Quantité vendue
LigneVente	prix_unitaire	Prix au moment de la vente (snapshot)
LigneVente	sous_total	Calculé: quantité × prix_unitaire

Note métier : Le prix_unitaire dans LigneVente doit être un snapshot du prix au moment de la vente. Ne faites pas de ForeignKey au prix — les prix peuvent changer dans le temps.

4.3 Catégories de Médicaments

Gérer une liste de catégories pour classifier les médicaments.

- CRUD complet sur les catégories
- Chaque catégorie a un nom et une description optionnelle
- Un médicament appartient à une catégorie

© 2026 SMARTHOLOL — Tous droits réservés

5. Spécifications Techniques

5.1 Structure du Projet Backend

La structure de votre projet Django doit être claire et professionnelle. Voici la structure attendue :

```
pharma_backend/
├── config/                    # Dossier de configuration du projet
│   ├── __init__.py
│   ├── settings/
│   │   ├── base.py            # Settings communs
│   │   └── local.py           # Settings de développement
│   ├── urls.py                # URLs racine
│   └── wsgi.py
├── apps/
│   ├── medicaments/           # App gestion médicaments
│   │   ├── models.py
│   │   ├── serializers.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   └── admin.py
│   ├── ventes/                # App gestion des ventes
│   │   ├── models.py
│   │   ├── serializers.py
│   │   ├── views.py
│   │   └── urls.py
│   └── categories/            # App catégories
├── .env.example               # Variables d'environnement (sans valeurs sensibles)
├── .gitignore
├── requirements.txt
├── manage.py
└── README.md
```

5.2 Structure du Projet Frontend

```
pharma_frontend/
├── src/
│   ├── api/                   # Couche d'accès aux données
│   │   ├── axiosConfig.js     # Instance Axios configurée
│   │   ├── medicamentsApi.js  # Fonctions API médicaments
│   │   └── ventesApi.js       # Fonctions API ventes
│   ├── components/            # Composants réutilisables
│   │   ├── common/            # Boutons, inputs, modals...
│   │   ├── medicaments/       # Composants spécifiques aux médicaments
│   │   └── ventes/            # Composants spécifiques aux ventes
│   ├── pages/                 # Pages de l'application
│   │   ├── MedicamentsPage.jsx
│   │   ├── VentesPage.jsx
```

© 2026 SMARTHOLOL — Tous droits réservés

```
│   │   └── DashboardPage.jsx
│   ├── hooks/                  # Custom React hooks
│   ├── utils/                  # Fonctions utilitaires
│   └── App.jsx
├── .env.example
├── .gitignore
└── README.md
```

5.3 Exigences Clean Code

> *Le clean code n'est pas optionnel chez SMARTHOLOL. Voici les pratiques obligatoires que nous attendons dans votre rendu.*

Backend Python/Django :

- Docstrings sur chaque classe de modèle, serializer et viewset
- Nommage en snake_case pour variables et fonctions, PascalCase pour classes
- Chaque view ou viewset doit avoir une responsabilité unique et claire
- Utiliser les ViewSets DRF plutôt que des views fonctionnelles basiques
- Les serializers doivent valider les données (validate_<field> ou validate())
- Gestion des erreurs explicite avec les bons status codes HTTP (400, 404, 422...)
- Aucun secret dans le code — utiliser les variables d'environnement (.env)

Frontend React :

- Séparation claire : les composants UI ne font pas d'appels API directement
- Tous les appels API passent par le dossier api/
- Gestion des états de chargement (loading) et d'erreur pour chaque requête
- Composants courts et focalisés (< 150 lignes idéalement)
- Utiliser les custom hooks pour la logique réutilisable
- Variables et composants nommés clairement (PascalCase pour composants)

5.4 Exigences API & Swagger

Endpoints minimaux attendus :

Méthode	Endpoint	Description
GET	/api/v1/medicaments/	Liste paginée des médicaments actifs
POST	/api/v1/medicaments/	Créer un nouveau médicament
GET	/api/v1/medicaments/{id}/	Détail d'un médicament
PUT/PATCH	/api/v1/medicaments/{id}/	Modifier un médicament
DELETE	/api/v1/medicaments/{id}/	Soft delete (est_actif=False)

© 2026 SMARTHOLOL — Tous droits réservés

Méthode	Endpoint	Description
GET	/api/v1/medicaments/alertes/	Médicaments sous stock minimum
GET	/api/v1/categories/	Liste des catégories
POST	/api/v1/categories/	Créer une catégorie
GET	/api/v1/ventes/	Historique des ventes
POST	/api/v1/ventes/	Enregistrer une vente (avec lignes)
GET	/api/v1/ventes/{id}/	Détail d'une vente
POST	/api/v1/ventes/{id}/annuler/	Annuler une vente

Swagger : Chaque endpoint doit avoir une description, des tags de regroupement, des exemples de réponse, et les codes d'erreur documentés. Utilisez drf-spectacular et configurez correctement @extend_schema.

5.5 Exigences Git

Le dépôt Git est un indicateur majeur de votre façon de travailler en équipe. Voici nos attentes :

Structure du dépôt :

- Un repository unique avec deux dossiers : backend/ et frontend/
- Un .gitignore complet (pas de venv/, node_modules/, .env dans le repo)
- Un README.md clair à la racine avec instructions d'installation

Convention de commits (Conventional Commits) :

```
feat: add medicament list endpoint
feat: create sale with stock deduction
fix: correct stock update on sale cancellation
docs: add swagger schemas for ventes endpoints
refactor: extract stock validation to service layer
chore: add .env.example and update gitignore
```

Ne faites pas de commit unique avec tout votre code. Committez au fur et à mesure, comme vous le feriez dans un vrai projet d'équipe. Nous regarderons votre historique de commits.

© 2026 SMARTHOLOL — Tous droits réservés

6. Livrables Attendus

6.1 Livrable Backend

1. Projet Django configuré avec PostgreSQL et les variables d'environnement
2. Modèles avec migrations propres et données de test (fixtures ou seeds)
3. API REST complète pour Médicaments, Catégories et Ventes
4. Documentation Swagger accessible sur /api/schema/swagger-ui/
5. Fichier .env.example avec toutes les variables nécessaires
6. README.md avec instructions d'installation pas à pas

6.2 Livrable Frontend

1. Page Médicaments : liste avec filtres, formulaire d'ajout/modification, indicateurs de stock bas
2. Page Ventes : formulaire de création d'une vente avec sélection de médicaments et quantités
3. Dashboard simple : nombre de médicaments, alertes de stock, ventes du jour
4. Gestion des erreurs API avec messages utilisateur clairs
5. Configuration Axios avec baseURL depuis variables d'environnement

6.3 Livrable Git

1. Lien vers le repository GitHub/GitLab (public ou accès partagé)
2. Historique de commits lisible et structuré (10+ commits minimum)
3. README.md complet à la racine du projet

© 2026 SMARTHOLOL — Tous droits réservés

7. Grille d'Évaluation

Votre rendu sera évalué selon les critères suivants. Chaque critère est noté indépendamment.

Critère	Points	Détail de l'évaluation
Modélisation BDD	20	Relations correctes, nommage, contraintes, migrations propres
API REST & Logique métier	20	Endpoints complets, logique stock, gestion erreurs, status codes
Clean Code Backend	15	Docstrings, nommage, structure, séparation responsabilités
Documentation Swagger	15	Couverture complète, descriptions, exemples, schemas
Frontend React	15	Structure composants, appels API, gestion état/erreur
Git & README	10	Commits conventionnels, .gitignore, README d'installation
Clean Code Frontend	5	Séparation api/components, hooks, lisibilité
TOTAL	100	

© 2026 SMARTHOLOL — Tous droits réservés

8. Guide de Démarrage & Bonnes Pratiques

> *Cette section vous guide pour démarrer le projet correctement. Elle reflète exactement comment nous initialisons nos projets chez SMARTHOLOL. Lisez-la avant de commencer.*

8.1 Initialisation du Backend Django

```
# 1. Créer et activer l'environnement virtuel
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate    # Windows

# 2. Installer les dépendances
pip install django djangorestframework drf-spectacular
pip install psycopg2-binary python-decouple pillow
pip freeze > requirements.txt

# 3. Créer le projet et les apps
django-admin startproject config .
python manage.py startapp medicaments
python manage.py startapp ventes
python manage.py startapp categories

# 4. Configurer settings.py (utiliser python-decouple pour .env)
# 5. Faire les migrations
python manage.py makemigrations
python manage.py migrate
```

8.2 Configuration Swagger (drf-spectacular)

Voici la configuration minimale pour activer Swagger dans votre projet :

```
# settings.py
INSTALLED_APPS = [
    ...
    'rest_framework',
    'drf_spectacular',
]

REST_FRAMEWORK = {
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
}

SPECTACULAR_SETTINGS = {
    'TITLE': 'PharmaManager API',
    'DESCRIPTION': 'API de gestion de pharmacie — Développé avec SMARTHOLOL
standards',
```

© 2026 SMARTHOLOL — Tous droits réservés

```
    'VERSION': '1.0.0',
    'SERVE_INCLUDE_SCHEMA': False,
}

# urls.py
from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView

urlpatterns = [
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
    path('api/schema/swagger-ui/',
SpectacularSwaggerView.as_view(url_name='schema'), name='swagger-ui'),
]
```

8.3 Exemple d'un Modèle Bien Documenté

Voici comment nous écrivons nos modèles chez SMARTHOLOL :

```
class Medicament(models.Model):
    """
    Représente un médicament dans l'inventaire de la pharmacie.

    Attributs:
        nom (str): Nom commercial du médicament.
        stock_actuel (int): Quantité disponible en stock.
        stock_minimum (int): Seuil déclenchant une alerte de réapprovisionnement.
        est_actif (bool): Soft delete. False = médicament archivé.
    """
    nom = models.CharField(max_length=200, verbose_name='Nom commercial')
    dci = models.CharField(max_length=200, verbose_name='DCI', blank=True)
    categorie = models.ForeignKey(
        'categories.Categorie',
        on_delete=models.PROTECT,
        related_name='medicaments',
        verbose_name='Catégorie'
    )
    stock_actuel = models.PositiveIntegerField(default=0)
    stock_minimum = models.PositiveIntegerField(default=10)
    est_actif = models.BooleanField(default=True, verbose_name='Actif')
    date_creation = models.DateTimeField(auto_now_add=True)

    class Meta:
        verbose_name = 'Médicament'
        verbose_name_plural = 'Médicaments'
        ordering = ['nom']

    def __str__(self):
        return f'{self.nom} ({self.dosage})'

    @property
    def est_en_alerte(self):
        """Retourne True si le stock est inférieur au seuil minimum."""
        return self.stock_actuel <= self.stock_minimum
```

© 2026 SMARTHOLOL — Tous droits réservés

8.4 Exemple d'un Appel API Propre en React

```
// src/api/medicamentsApi.js

import axiosInstance from './axiosConfig';

/**
 * Récupère la liste paginée des médicaments.
 * @param {Object} params - Paramètres de filtrage (search, categorie, page)
 * @returns {Promise<Object>} Données paginées des médicaments
 */
export const fetchMedicaments = async (params = {}) => {
  const response = await axiosInstance.get('/medicaments/', { params });
  return response.data;
};

/**
 * Crée un nouveau médicament.
 * @param {Object} data - Données du médicament
 */
export const createMedicament = async (data) => {
  const response = await axiosInstance.post('/medicaments/', data);
  return response.data;
};

// src/hooks/useMedicaments.js
import { useState, useEffect } from 'react';
import { fetchMedicaments } from '../api/medicamentsApi';

export const useMedicaments = (filters = {}) => {
  const [medicaments, setMedicaments] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  useEffect(() => {
    const loadData = async () => {
      setLoading(true);
      try {
        const data = await fetchMedicaments(filters);
        setMedicaments(data.results);
      } catch (err) {
        setError(err.response?.data?.message || 'Une erreur est survenue');
      } finally {
        setLoading(false);
      }
    };
    loadData();
  }, []);

  return { medicaments, loading, error };
};
```

© 2026 SMARTHOLOL — Tous droits réservés

9. Template README.md

Votre README doit être professionnel et permettre à n'importe quel développeur de lancer le projet en moins de 10 minutes. Voici la structure attendue :

```
# PharmaManager
Application de gestion de pharmacie — Développé dans le cadre du test technique
SMARTHOLOL

## Stack Technique
- Backend : Django 4.x + Django REST Framework + PostgreSQL
- Frontend : React.js (Vite)
- Documentation API : Swagger (drf-spectacular)

## Installation Backend
```bash
cd backend
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env  # Configurer les variables
python manage.py migrate
python manage.py loaddata fixtures/initial_data.json  # Données de test
python manage.py runserver
```

Variables d'Environnement (.env)
```
DEBUG=True
SECRET_KEY=votre-secret-key
DB_NAME=pharma_db
DB_USER=postgres
DB_PASSWORD=password
DB_HOST=localhost
DB_PORT=5432
```

Installation Frontend
```bash
cd frontend
npm install
cp .env.example .env
npm run dev
```

Documentation API
Swagger UI disponible sur : http://localhost:8000/api/schema/swagger-ui/
```

© 2026 SMARTHOLOL — Tous droits réservés

10. Checklist de Soumission

Avant de soumettre votre travail, vérifiez chaque point de cette liste :

Backend

☐	Les modèles sont créés avec toutes les relations et contraintes
☐	Les migrations sont propres et sans conflit
☐	Tous les endpoints CRUD fonctionnent (testé via Swagger ou Postman)
☐	L'endpoint /alertes/ retourne les médicaments sous le seuil minimum
☐	La création d'une vente déduit bien les stocks
☐	L'annulation d'une vente réintègre bien les stocks
☐	Swagger est accessible et documenté
☐	Aucune valeur sensible n'est dans le code (SECRET_KEY, mot de passe...)
☐	requirements.txt est à jour
☐	.env.example est présent avec toutes les variables

Frontend

☐	La liste des médicaments s'affiche correctement
☐	Les indicateurs de stock bas sont visibles
☐	Le formulaire d'ajout de médicament fonctionne
☐	La création d'une vente fonctionne et met à jour le stock
☐	Les états loading et error sont gérés pour chaque requête API
☐	L'URL de l'API est dans une variable d'environnement (pas en dur)

Git & Documentation

☐	Le repository est accessible (public ou partagé)
☐	Le .gitignore est complet (venv/, node_modules/, .env, __pycache__/)
☐	Les commits sont clairs, conventionnels, et au moins 10
☐	Le README contient les instructions d'installation complètes

© 2026 SMARTHOLOL — Tous droits réservés

11. Points Bonus (Optionnel)

Important : Les points bonus ne compensent pas un manque sur les critères principaux. Finissez les exigences de base avant de vous attaquer aux bonus.

Fonctionnalité Bonus	Points bonus
Tests unitaires sur les views et serializers (Django TestCase)	+10 pts
Authentification JWT (djangorestframework-simplejwt)	+8 pts
Filtres avancés avec django-filter (par catégorie, date, statut)	+5 pts
Pagination personnalisée avec métadonnées enrichies	+3 pts
Export CSV de l'inventaire ou des ventes	+5 pts
Docker Compose pour lancer le projet en une commande	+7 pts

© 2026 SMARTHOLOL — Tous droits réservés

12. Soumission et Contact

Une fois votre travail terminé, envoyez le lien de votre repository Git ainsi que votre CV à l'équipe technique de SMARTHOLOL via le formulaire suivant. Assurez-vous que le repository est accessible en **public** avant l'envoi.

Lien : https://nocodb.smartholol.com/dashboard/#/nc/form/71ebe10c-259b-4b89-9485-d6e380a52e54

Ce que nous évaluerons en priorité

Votre code nous révèle ce que vous pensez. Un projet avec moins de fonctionnalités, mais un code propre, bien structuré et bien documenté, sera toujours préférable à un projet complet mais désorganisé. Chez SMARTHOLOL, nous travaillons avec des clients exigeants dans des environnements multi-pays. La rigueur et la professionnalisme ne sont pas des options.

SMARTHOLOL — Équipe Technique
AI Solutions | Automation | Custom Development
Maroc | USA | Australie | Canada
contact@smartholol.com

© 2026 SMARTHOLOL — Tous droits réservés